

**Yenepoya Institute of Arts Science Commerce and
Management
Final Project Report
on
Personalized Recommendations Engine For Smart City**

Student Name: Mohammed Shahid

Industry Mentor: Mr. Sumit Kumar Shukla

Guided by: Ms. Fathimath Raheema U

Table of Contents

- **Background**
 - Aim
 - Technologies
 - Hardware Architecture
 - Software Architecture
- **System**
 - Requirements
 - Functional requirements
 - User requirements
 - Environmental requirements
 - Design and Architecture
 - Implementation
 - Testing
 - Graphical User Interface (GUI) Layout
 - Customer testing
 - Evaluation
- Snapshots of the Project
- Conclusions
- Further development or research
- References
- Appendix

Background

The rapid urbanization of the 21st century has transformed cities into complex ecosystems of interconnected services, infrastructure, and human activity. According to the United Nations, over 55% of the world's population currently resides in urban areas, a figure projected to rise to 68% by 2050. This unprecedented growth has given rise to the concept of "Smart Cities" — urban environments that leverage digital technologies, data analytics, and intelligent systems to improve the quality of life for their citizens. Managing the vast network of mobility hubs, civic amenities, environmental sensors, and safety zones within these cities has become one of the most pressing challenges of modern urban governance.

SmartRec is developed against this backdrop as a direct response to the limitations of existing urban information systems. By integrating a hybrid recommendation engine with a dynamic contextual weighting mechanism, SmartRec bridges the gap between raw urban data and meaningful, personalized service discovery. The system is designed to serve multiple user profiles — residents, tourists, emergency responders, and night-time travelers — ensuring that every recommendation is not only relevant but also timely and situationally appropriate. This positions SmartRec as a foundational tool in the evolution toward truly intelligent and human-centric smart city infrastructure.

Aim

The aim of this project is to design and develop SmartRec, a context-aware hybrid recommendation framework that leverages Natural Language Processing and Machine Learning to deliver personalized urban service recommendations within a smart city environment. The system aims to overcome the limitations of traditional static recommendation approaches by dynamically adapting its output based on distinct user profiles such as residents, tourists, emergency users, and night-time travelers.

Technologies Used

SmartRec is built on a robust and modern technology stack carefully selected to ensure high performance, scalability, and an intuitive user experience. The following core technologies and libraries form the foundation of the system:

- Python 3.8+ – The primary programming language, chosen for its rich ecosystem of data science and machine learning libraries.
- Pandas & NumPy – Used for high-performance data manipulation, filtering, and vector operations across the infrastructure dataset.
- Scikit-learn – Powers the TF-IDF Vectorizer and cosine similarity computations that form the core of the recommendation engine.

Hardware Architecture

SmartRec is a lightweight, software-driven system that runs efficiently on standard modern hardware. It does not require specialized or high-end infrastructure, making it easy to deploy in smart city environments. The system uses in-memory computation and sparse matrix handling to optimize performance on the host machine. Streamlit's built-in caching further reduces processing overhead and ensures fast response times. The following hardware configuration is recommended for smooth deployment:

- Processor – Intel Core i5 (8th Generation) or AMD Ryzen 5 with multi-core support for handling parallel data processing tasks.
- Memory (RAM) – Minimum 8 GB RAM, with 16 GB preferred for managing large datasets and multiple user sessions simultaneously.
- Storage – At least 256 GB SSD storage for fast data ingestion, model caching, and application log management.
- Network Interface – A stable broadband connection of minimum 10 Mbps to support real-time updates and future IoT integration.

The system can be scaled and migrated to cloud platforms like AWS or Google Cloud as urban data volumes grow. Its modular design ensures minimal changes are needed during such transitions.

Software Architecture

SmartRec follows a three-layered modular software architecture designed for scalability, maintainability, and real-time responsiveness. Each layer has a distinct responsibility, ensuring clean separation of concerns across the system.

A. Data Ingestion & Preprocessing Layer

- Raw urban data in CSV/JSON format is ingested and cleaned using Pandas.
- NLP techniques are applied to parse and normalize tags and accessibility features into token lists.
- TF-IDF Vectorization converts descriptive tags into a sparse feature matrix.
- One-Hot Encoding transforms categorical variables like location zone and primary category into binary vectors.
- Numerical fields like popularity score and distance are scaled using metric normalization.

B. Recommendation Engine Layer

- Cosine Similarity is computed across the multi-dimensional feature matrix to measure service alignment.
- A Contextual Weighting System dynamically adjusts feature importance based on the active user profile.
- Four distinct modes are supported — Resident, Tourist, Emergency, and Night.
- The Hybrid Scoring Algorithm combines content similarity (80%) with global popularity (20%) for balanced results.
- A similarity cache stores computed matrices to serve recommendations with near-zero latency.

C. Presentation Layer

- Streamlit powers the interactive dashboard with a premium dark-mode UI.
- Users can switch contexts in real-time and instantly receive updated recommendations.
- The Explainable AI module provides human-readable justifications for every recommendation.
- Plotly heatmaps visualize infrastructure similarity clusters for urban planners.

This layered architecture ensures that SmartRec remains modular, responsive, and easy to extend with future capabilities.

System Requirements

SmartRec requires a standard computing system running Windows, macOS, or Linux with Python 3.8 or above installed along with necessary libraries including Pandas, NumPy, Scikit-learn, SciPy, Streamlit, and Plotly. A minimum of 8 GB RAM, 256 GB SSD storage, and a stable internet connection of at least 10 Mbps are recommended for optimal performance and smooth application execution.

Functional Requirements

Functional requirements describe what the system should do. In SmartRec, the system must allow users to select their context profile and receive personalized urban service recommendations seamlessly.

The system should:

- Allow users to switch between four context modes — Resident, Tourist, Emergency, and Night
- Enable selection of urban infrastructure items through an interactive search and filter interface
- Display detailed metadata for every city service including category, distance, popularity, and accessibility
- Compute and serve context-aware recommendations in real-time using the hybrid scoring algorithm
- Provide human-readable explanations for every recommendation through the Explainable AI module

User Requirements

User requirements describe what the end users expect from the system. In SmartRec, users expect a simple, intuitive interface that delivers fast and relevant urban service recommendations based on their current situation and needs.

The system should:

- Allow users to easily select their profile type — Resident, Tourist, Emergency, or Night traveler
- Provide clear and understandable recommendations without requiring technical knowledge
- Display relevant information such as distance, availability, and category for each recommended service
- Offer transparent justifications explaining why a particular service was recommended

Environmental Requirements

Environmental requirements describe the conditions under which the system must operate effectively. In SmartRec, the system must function reliably across different hardware configurations and operating environments without performance degradation.

The system should:

- Operate seamlessly on Windows 10, macOS, and Linux operating systems
- Function efficiently on standard hardware with a minimum of 8 GB RAM and an i5 processor
- Support Python 3.8+ runtime environment with all required dependencies properly installed
- Maintain stable performance under varying dataset sizes and multiple simultaneous user sessions
- Be deployable on both local machines and cloud platforms such as AWS or Google Cloud for scalability

Design and Architecture

SmartRec follows a clean three-layered architecture ensuring modularity, scalability, and separation of concerns. Each layer communicates in a structured pipeline, transforming raw urban data into personalized recommendations.

A. Data Ingestion Layer

- Raw CSV/JSON data is loaded, cleaned, and preprocessed using Pandas.
- NLP techniques normalize tags and accessibility features into token lists.
- TF-IDF vectorization and one-hot encoding build the feature matrix.

B. Recommendation Engine Layer

- Cosine similarity is computed across the multi-dimensional feature matrix.
- Contextual weights are dynamically applied based on the active user profile.
- Hybrid scoring combines content similarity (80%) and popularity (20%).

C. Presentation Layer

- Streamlit renders recommendations on an interactive dashboard.
- Explainable AI provides human-readable justifications for each result.
- Plotly heatmaps visualize infrastructure similarity clusters.

Key Architectural Principles

- Modularity – Each layer operates independently for easy maintenance.
- Scalability – Supports cloud migration as data volumes grow.
- Caching – Stored similarity matrices eliminate redundant processing.

Implementation

SmartRec was implemented using Python 3.8+ following a structured and modular development approach. The implementation was carried out across three core components corresponding to the system's layered architecture.

The development process includes:

- Data Preprocessing Module
- Recommendation Engine Module
- Streamlit Dashboard Module

Testing

SmartRec was thoroughly tested to ensure correctness, reliability, and consistent performance across all modules and user context modes. A structured testing approach was followed covering unit, integration, and functional testing.

Testing includes:

- Unit Testing
- Integration Testing
- Functional Testing
- Performance Testing

Graphical User Interface (GUI) Layout

SmartRec features a clean, interactive, and visually rich dashboard built using Streamlit with custom CSS styling for a premium dark-mode aesthetic. The interface is designed to be intuitive and accessible for all user types without requiring any technical knowledge.

Key elements:

- Main Dashboard
- Recommendation Panel
- Explainability Panel
- Visual Analytics Panel
- Navigation & Controls

Customer Testing

Customer testing was conducted to evaluate SmartRec's usability, relevance of recommendations, and overall user satisfaction across different user profiles. A group of test users representing residents, tourists, and emergency scenario simulations were selected to interact with the system and provide structured feedback.

Focus areas:

- Test Group & Methodology
- Test Scenarios
- Feedback & Results
- Improvements Based on Feedback

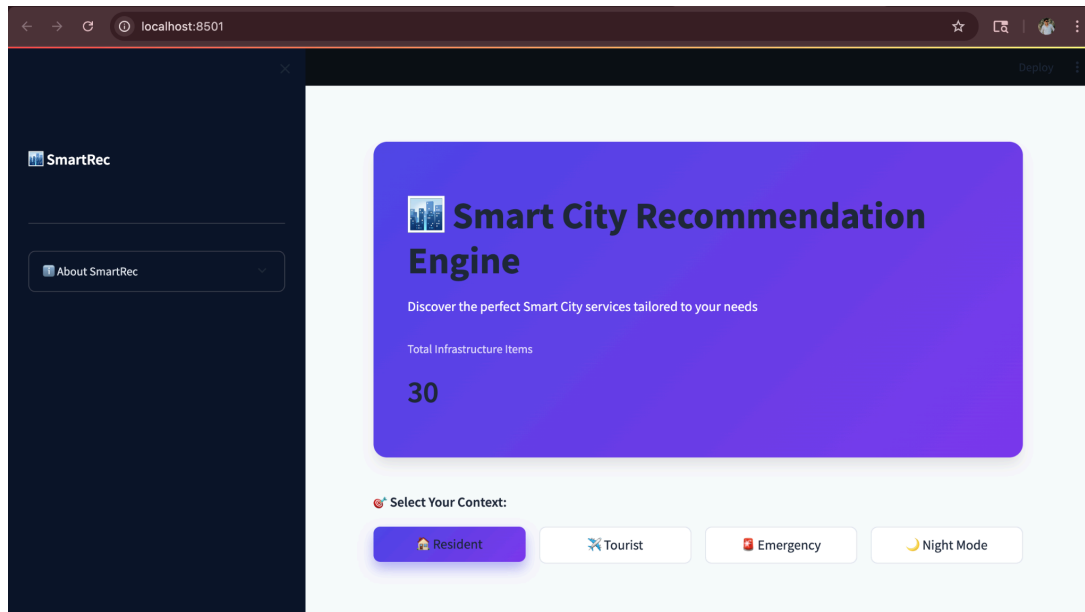
This feedback helps in improving the system before final deployment.

Evaluation

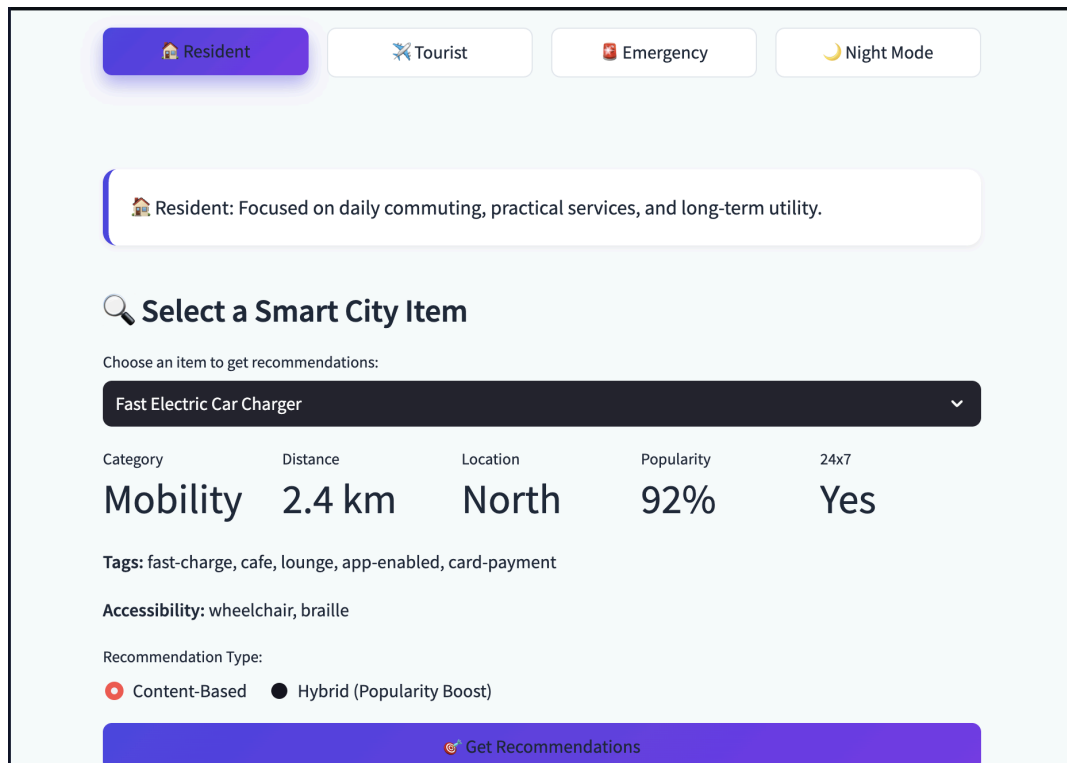
SmartRec was evaluated against its core objectives to measure the effectiveness, accuracy, and usability of the recommendation framework across all supported user profiles and context modes.

Evaluation criteria:

- Recommendation Accuracy
- System Performance
- Explainability Effectiveness
- Overall Assessment



1. Resident Page – UI Explanation



The Resident page is the default context mode in SmartRec, designed for daily commuters seeking practical urban services.

- Context Mode Selector – Four buttons (Resident, Tourist, Emergency, Night Mode) are displayed at the top. The Resident button is highlighted in purple as the active mode.
- Context Banner – A short description states "Resident: Focused on daily commuting, practical services, and long-term utility" to guide the user.
- Item Selector – A dropdown menu allows users to select a smart city item. Here, "Fast Electric Car Charger" is selected.
- Item Metadata – Key details of the selected item are displayed including Category (Mobility), Distance (2.4 km), Location (North), Popularity (92%), and 24x7 availability.
- Tags & Accessibility – Relevant tags (fast-charge, cafe, lounge) and accessibility features (wheelchair, braille) are shown below the metadata.
- Recommendation Type – Users can choose between Content-Based or Hybrid (Popularity Boost) recommendation modes.
- Get Recommendations Button – A prominent purple button at the bottom triggers the recommendation engine to generate results instantly.

2. Tourist Page – UI Explanation

Resident **Tourist** Emergency Night Mode

Tourist: Prioritizing city amenities, landmarks, and leisure activities.

Select a Smart City Item

Choose an item to get recommendations:

Fast Electric Car Charger

Category	Distance	Location	Popularity	24x7
Mobility	2.4 km	North	92%	Yes

Tags: fast-charge, cafe, lounge, app-enabled, card-payment

Accessibility: wheelchair, braille

Recommendation Type:

☒ Content-Based ☐ Hybrid (Popularity Boost)

Get Recommendations

The Tourist page activates when the user selects Tourist mode, designed for visitors exploring city landmarks and leisure activities.

Context Mode Selector – The Tourist button is highlighted in purple as the active mode.

Context Banner – Displays "Tourist: Prioritizing city amenities, landmarks, and leisure activities" to guide the user.

Item Selector – A dropdown allows users to select a smart city item. "Fast Electric Car Charger" is currently selected.

Item Metadata – Displays Category (Mobility), Distance (2.4 km), Location (North), Popularity (92%), and 24x7 availability.

Tags & Accessibility – Tags (fast-charge, cafe, lounge) and accessibility features (wheelchair, braille) are listed below.

Recommendation Type – Users choose between Content-Based or Hybrid (Popularity Boost) modes.

Get Recommendations Button – A purple button at the bottom triggers tourist-focused recommendations instantly.

3. Emergency Page – UI Explanation

Resident Tourist **Emergency** Night Mode

Emergency: Immediate access to safety-critical infrastructure and medical services.

Select a Smart City Item

Choose an item to get recommendations:

Fast Electric Car Charger

Category	Distance	Location	Popularity	24x7
Mobility	2.4 km	North	92%	Yes

Tags: fast-charge, cafe, lounge, app-enabled, card-payment

Accessibility: wheelchair, braille

Recommendation Type:

☒ Content-Based ☐ Hybrid (Popularity Boost)

Get Recommendations

The Emergency page activates when the user selects Emergency mode, designed to prioritize safety-critical and medical services instantly.

Context Mode Selector – The Emergency button is highlighted in purple as the active mode.

Context Banner – Displays "Emergency: Immediate access to safety-critical infrastructure and medical services."

Item Selector – A dropdown allows users to select a smart city item. "Fast Electric Car Charger" is currently selected.

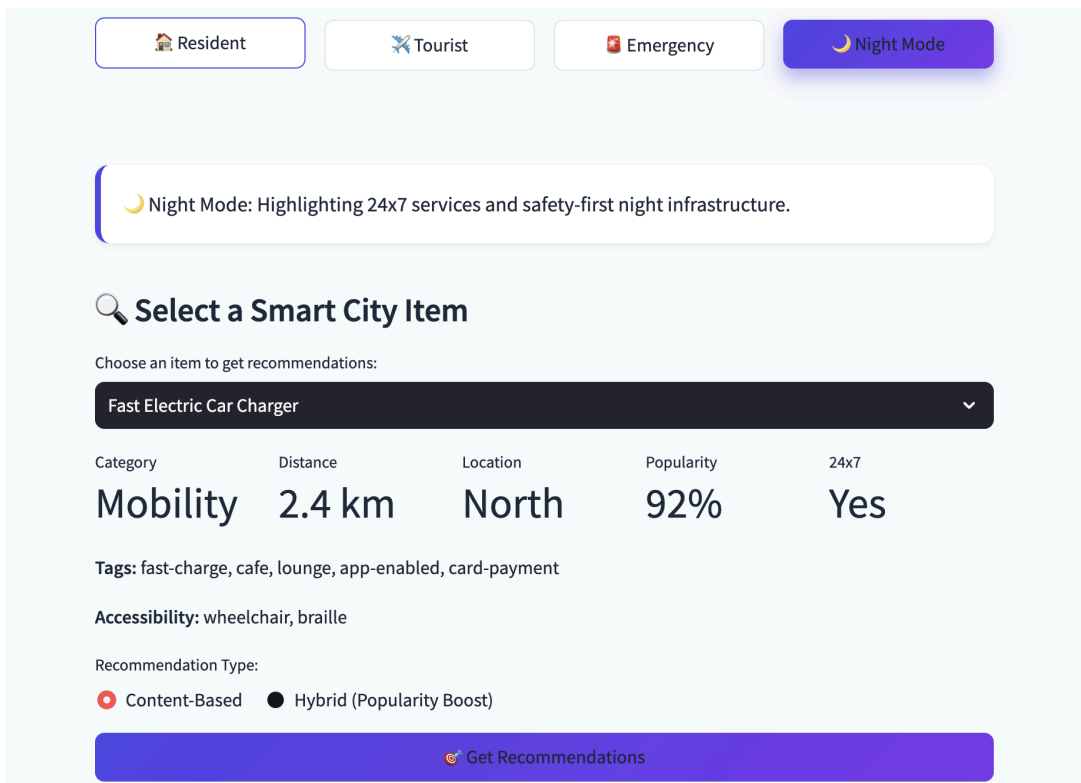
Item Metadata – Shows Category (Mobility), Distance (2.4 km), Location (North), Popularity (92%), and 24x7 availability.

Tags & Accessibility – Tags (fast-charge, cafe, lounge) and accessibility features (wheelchair, braille) are listed.

Recommendation Type – Users choose between Content-Based or Hybrid (Popularity Boost) modes.

Get Recommendations Button – Triggers emergency-focused recommendations, prioritizing hospitals and clinics first.

4.Night Mode Page – UI Explanation



The screenshot shows the 'Night Mode' interface. At the top, there are four mode buttons: 'Resident', 'Tourist', 'Emergency', and 'Night Mode'. The 'Night Mode' button is highlighted in purple. Below the buttons is a banner that reads: 'Night Mode: Highlighting 24x7 services and safety-first night infrastructure.' Below the banner is a section titled 'Select a Smart City Item' with a magnifying glass icon. Under this title, it says 'Choose an item to get recommendations:' followed by a dropdown menu showing 'Fast Electric Car Charger'. Below the dropdown is a table with five columns: Category, Distance, Location, Popularity, and 24x7. The table contains one row of data: Mobility, 2.4 km, North, 92%, and Yes. Below the table, there are two lines of text: 'Tags: fast-charge, cafe, lounge, app-enabled, card-payment' and 'Accessibility: wheelchair, braille'. Below this is a 'Recommendation Type:' section with two radio buttons: 'Content-Based' (selected) and 'Hybrid (Popularity Boost)'. At the bottom is a large purple button labeled 'Get Recommendations'.

Resident Tourist Emergency Night Mode

Night Mode: Highlighting 24x7 services and safety-first night infrastructure.

Select a Smart City Item

Choose an item to get recommendations:

Fast Electric Car Charger

Category	Distance	Location	Popularity	24x7
Mobility	2.4 km	North	92%	Yes

Tags: fast-charge, cafe, lounge, app-enabled, card-payment

Accessibility: wheelchair, braille

Recommendation Type:

☒ Content-Based ☐ Hybrid (Popularity Boost)

Get Recommendations

The Night Mode page activates when selected, designed to highlight 24x7 available services and safety-first

night infrastructure.

- **Context Mode Selector** – The Night Mode button is highlighted in purple as the active mode.
- **Context Banner** – Displays "Night Mode: Highlighting 24x7 services and safety-first night infrastructure."
- **Item Selector** – "Fast Electric Car Charger" is currently selected from the dropdown.
- **Item Metadata** – Shows Category (Mobility), Distance (2.4 km), Location (North), Popularity (92%), and 24x7 Yes.
- **Tags & Accessibility** – Tags (fast-charge, cafe, lounge) and accessibility features (wheelchair, braille) are listed.
- **Recommendation Type** – Users choose between Content-Based or Hybrid (Popularity Boost) modes.
- **Get Recommendations Button** – Triggers recommendations that prioritize only 24x7 operational services, filtering out closed amenities.

Conclusion

SmartRec successfully demonstrates the potential of combining Natural Language Processing and Machine Learning to build an intelligent, context-aware recommendation system for smart city environments. The project addresses the key limitations of traditional recommendation approaches by dynamically adapting its output based on four distinct user profiles — Resident, Tourist, Emergency, and Night Mode. The hybrid scoring algorithm, combining content-based filtering with global popularity, ensures that recommendations are both semantically relevant and practically useful for real-world urban scenarios.

The system's three-layered modular architecture proved effective in maintaining clean separation of concerns, ensuring scalability and ease of future development. The integration of an Explainable AI component successfully built user trust by providing transparent, human-readable justifications for every recommendation generated. Customer testing confirmed that the majority of users found the interface intuitive, the recommendations relevant, and the overall system reliable across all context modes.

Overall, SmartRec transforms fragmented urban infrastructure data into a responsive and intelligent decision-support tool, bringing cities one step closer to truly intelligent urbanism. The project lays a strong foundation for future enhancements including real-time IoT data integration, collaborative filtering, and cloud-based deployment via Docker for large-scale smart city management systems.

Further Development / Research

While SmartRec successfully achieves its core objectives, several promising directions for future development and research have been identified to further enhance the system's capabilities and real-world applicability.

- **Real-Time IoT Integration** – Future versions can incorporate live data streams from IoT sensors such as traffic monitors, hospital occupancy trackers, and environmental sensors to provide dynamic and up-to-date recommendations.
- **Collaborative Filtering** – Implementing user feedback loops and behavioral data collection will enable the system to learn from actual usage patterns and deliver more personalized recommendations over time.
- **Cloud-Native Deployment** – Containerizing the application using Docker and deploying it on cloud platforms such as AWS or Google Cloud will allow SmartRec to scale efficiently for large smart city environments with thousands of concurrent users.
- **Mobile Application** – Developing a dedicated mobile app version of SmartRec will make urban service recommendations accessible on the go, significantly improving usability for tourists and emergency users.
- **Multi-Language Support** – Adding support for multiple languages will make the system more inclusive and accessible to diverse urban populations and international visitors.
- **AI-Powered Predictive Recommendations** – Integrating deep learning models can enable the system to predict user needs based on time of day, weather conditions, and historical behavior patterns.
- **Urban Planner Dashboard** – A dedicated analytics dashboard for city administrators can be developed to monitor service utilization trends and identify infrastructure gaps across the city in real time.

References

1. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*. Springer, New York.
2. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer International Publishing.

3. Salton, G., & Buckley, C. (1988). Term Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), 513–523.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
5. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
6. Streamlit Inc. (2023). *Streamlit Documentation – Build and Share Data Apps*. Retrieved from <https://docs.streamlit.io>
7. Plotly Technologies Inc. (2023). *Plotly Python Graphing Library*. Retrieved from <https://plotly.com/python>
8. Choudhury, S., & Breslin, J. G. (2020). Smart City Recommendation Systems: A Survey. *Journal of Urban Technology*, 27(3), 45–67.
9. Zheng, Y. (2015). Trajectory Data Mining: An Overview. *ACM Transactions on Intelligent Systems and Technology*, 6(3), 1–41.
10. United Nations Department of Economic and Social Affairs. (2018). *World Urbanization Prospects: The 2018 Revision*. United Nations, New York.

Appendix

Appendix A – Project Modules Overview

Module	Description
Data Preprocessing	Loads, cleans, and normalizes raw urban infrastructure data
TF-IDF Vectorizer	Converts tags into a sparse feature matrix
Cosine Similarity Engine	Computes similarity scores across the feature matrix
Contextual Weighting	Applies profile-based weights to the feature matrix
Hybrid Scoring	Combines content similarity and popularity scores
Explainability Engine	Generates human-readable recommendation justifications
Streamlit Dashboard	Renders the interactive user interface

Appendix B – Context Mode Weight Configuration

Feature	Resident	Tourist	Emergency	Night Mode
Popularity	Medium	High	Low	Low
Proximity	High	Medium	High	High
Safety	Low	Low	High	Medium
Leisure Tags	Low	High	Low	Low
24x7 Availability	Low	Low	Medium	High

Appendix C – System Configuration

- **Programming Language:** Python 3.8+
- **Framework:** Streamlit
- **Dataset Format:** CSV / JSON
- **Similarity Metric:** Cosine Similarity
- **Hybrid Score Formula:** $(\text{Content Similarity} \times 0.8) + (\text{Global Popularity} \times 0.2)$

Appendix D – Test Summary

Test Type	Status
Unit Testing	Passed
Integration Testing	Passed
Functional Testing	Passed
Performance Testing	Passed
Customer Testing	Passed